

Chapter 49: Dependency Injection Internals

1. Overview

Every Angular application ships with a dependency injection engine that looks, from the outside, like a single call to `inject()` or a constructor parameter. Underneath, Ivy runs a two-tree resolution algorithm, a code generator that emits `ɵfac` and `ɵprov` static fields onto every injectable class, and a mutable "current injector" context that `inject()` reads synchronously and throws away the instant it's done. None of this uses `reflect-metadata`. None of it uses decorators at runtime to discover constructor parameter types — TypeScript's `emitDecoratorMetadata` is not required by modern Angular, and the whole system is closure-based and tree-shakeable.

This chapter dissects that machinery: how `ɵfac` factory functions are generated at compile time, how `NodeInjector` (the per-`LView` element injector) and `ModuleInjector` / `EnvironmentInjector` (the per-`NgModule` / standalone-injector hierarchy) form two separate but cooperating trees, how `inject()` reads a global injector-context stack rather than being "magically" scoped, how circular dependencies are detected and how `forwardRef` sidesteps them, and why `providedIn: 'root'` factories are registered lazily rather than eagerly at bootstrap.

The goal is to be able to answer, precisely, "what actually happens between `constructor(private http: HttpClient)` and having a working `HttpClient` instance in hand" — including which data structures are walked, in what order, and why a `NullInjector` sits at the top instead of `undefined`.

2. Core Concepts

2.1 Two trees, not one

Ivy DI is not a single injector hierarchy. It is **two parallel trees** that get stitched together at resolution time:

- **Element injector tree (`NodeInjector`)** — mirrors the DOM/component tree. Every component and every directive-bearing element *can* have injector data attached to its `LView` / `TView`, populated from providers declared in `@Component({ providers, viewProviders })` or `@Directive({ providers })`. This tree is walked by traversing `TNode.parent` pointers through the view hierarchy (including crossing into host views of embedded views / `ng-template` and `ViewContainerRef` boundaries).
- **Module/environment injector tree (`R3Injector`, exposed as `EnvironmentInjector`)** — mirrors module/standalone-injector nesting: platform injector → root `EnvironmentInjector` → any lazy-loaded feature `EnvironmentInjector`s (from lazy routes, `createEnvironmentInjector`, or `ApplicationConfig` providers for standalone apps). This tree is walked by following each `R3Injector`'s `.parent` reference.

A single logical injector lookup starts in the `NodeInjector` closest to the requesting component and, only when the entire element tree is exhausted, hands off to the *module injector associated with the component that owns the current view* — not to the root blindly. Each component's `LView` stores a reference to the `EnvironmentInjector` it was created under (important for lazy-loaded standalone components, each of which may have its own child `EnvironmentInjector`).

2.2 `ɵfac` — the factory function

For every class decorated with `@Injectable()`, `@Component()`, `@Directive()`, or `@Pipe()`, the Angular compiler (via `@angular/compiler-cli`, invoked by the AOT/JIT compiler paths) emits a static property:

```
MyService.ɵfac = function MyService_Factory(t) {  
  return new (t || MyService)(  
    ɵinject(Dep1),  
    ɵinject(Dep2, 8 /* optional flag */)  
  );  
};
```

This is generated **purely from the TypeScript type-checker's resolved constructor parameter types at compile time** — the compiler reads the constructor signature during compilation and bakes the token references directly into the factory as identifiers, not as strings or runtime-reflected metadata. That is the crux of "no reflect-metadata": the information that used to require `Reflect.getMetadata('design:paramtypes', MyService)` is captured statically by the compiler and turned into literal `ɵinject(Token)` calls in generated code. There is nothing to reflect at runtime because the calls are already there, hardcoded, ready to execute.

The `t` parameter supports inheritance: when a subclass doesn't redeclare its own constructor, Ivy reuses the parent's `ɵfac` but passes the subclass's constructor as `t` so `new (t || MyService)(...)` instantiates the *most derived* class while still injecting the *parent's* dependencies.

2.3 `ɵprov` — the provider definition

Separately, classes using `@Injectable({ providedIn: ... })` get a static `ɵprov`:

```
MyService.ɵprov = ɵdefineInjectable({  
  token: MyService,  
  factory: MyService.ɵfac,  
  providedIn: 'root'  
});
```

`ɵprov` is what makes a class **self-registering**: it doesn't need to appear in any `providers: []` array. Any injector, when it fails to find a provider for a token in its own records, checks whether the token itself carries a `ɵprov` with a matching `providedIn` scope and, if so, treats that as an implicit provider — this is what "tree-shakable providers" means mechanically.

2.4 `inject()` and the injection context

`inject(Token)` is not scoped by closures or by being "inside a constructor." It is scoped by a **module-level mutable variable** inside `packages/core/src/di/injector_compatibility.ts` — conceptually a single-slot global, `_currentInjector: Injector | undefined`, actually paired with an "injection context" object holding `{ injector, inCtx?, ...}`. Before invoking a factory function, Ivy calls `setCurrentInjector(injector)` (or `setInjectorProfilerContext` / `enterDI`-style helpers depending on version), runs the factory, and restores the previous value in a `finally` block — a manual save/restore stack discipline, not a real stack data structure, but it composes correctly because factory invocation is always synchronous and nested calls always restore before returning.

This is also why `inject()` throws `NG0203` ("inject() must be called from an injection context") when called outside: there's simply no injector currently installed in that slot at that moment — the check is `if (!currentInjector) throw new RuntimeError(...)`.

`runInInjectionContext(injector, fn)` is the public escape hatch: it manually performs the same save/set/run/restore dance so you can call `inject()` inside a `setTimeout`, an RxJS operator, or any other place that executes outside Angular's own factory-invocation call sites.

2.5 The resolution algorithm, in words

When `ɵinject(Token, flags)` executes:

1. If flags include `Self`, only the current `NodeInjector` level is checked — no walk.
2. Otherwise, walk up the `NodeInjector` chain (current element → ancestor elements → view boundaries), checking each `TNode`'s injector data for a matching provider, honoring `skipSelf` (skip the very first level) and `Host` (stop at the host boundary of the current component).
3. If nothing is found and `optional` isn't set to skip further, and the current node has an associated component with its own `EnvironmentInjector` (standalone components can have one), hand off there.
4. If the element tree is exhausted, hand off to the **module injector chain**: the `R3Injector` associated with the current view, then its `.parent`, up to the platform injector, then `NullInjector`.
5. At each `R3Injector`, if there's no explicit record for the token, check `Token.ɵprov.providedIn`: `'root'` matches the root `R3Injector`; `'platform'` matches the platform injector; `'any'` creates/returns a distinct instance per lazy-loaded injector; a module reference matches only that specific module's injector.
6. `NullInjector.get(token)` throws `NullInjectorError` (`NG0201`) unless `optional` was set, in which case `null` is returned.

2.6 providedIn: 'root' — lazy registration, not eager

The root `EnvironmentInjector` does **not** pre-populate a record for every `providedIn: 'root'` class at bootstrap. Instead, resolution step 5 above is the mechanism: the root injector's `get()` implementation, upon a miss in its own `records` map, inspects the token's `ɵprov.providedIn`. If it matches `'root'`, the injector calls the factory *right then*, caches the resulting instance into its own `records` map keyed by that token, and returns it. This is why an unused `providedIn: 'root'` service never gets instantiated (and can be tree-shaken entirely if nothing imports it) — the record is created on first demand, not at application start.

2.7 No reflect-metadata, mechanically

Pre-Ivy (View Engine) and non-Angular DI frameworks (e.g., InversifyJS) typically rely on TypeScript emitting `design:paramtypes` metadata via `emitDecoratorMetadata: true`, then read that array at runtime with `Reflect.getMetadata`. Angular's compiler instead performs **static analysis of the TypeScript AST** during compilation (`packages/compiler-cli`) and directly emits `ɵinject(ActualTokenReference)` calls into the `ɵfac` function body as compiled JavaScript. The dependency graph is fully resolved and hardcoded before the code ever runs, so:

- No `Reflect.getMetadata` calls exist in the runtime library at all.
- `emitDecoratorMetadata` is not a hard requirement for DI to work (it's occasionally needed for other

decorator-metadata use cases, but not for constructor injection).

- Minification/property-renaming doesn't break DI because tokens are referenced by identity (class reference/ `InjectionToken` object), never by string parameter names.

3. Code Examples

3.1 Sketch of a generated `ɵfac`/`ɵprov` pair

```
// Source:
@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(
    private http: HttpClient,
    @Optional() private logger: LoggerService,
    @Inject(APP_CONFIG) private config: AppConfig
  ) {}
}

// Roughly what the compiler emits (simplified, real output uses ɵɵ-prefixed
// runtime helpers and slightly different flag encoding per Ivy version):
UserService.ɵfac = function UserService_Factory(t) {
  return new (t || UserService)(
    ɵɵinject(HttpClient),
    ɵɵinject(LoggerService, 8 /* InjectFlags.Optional */),
    ɵɵinject(APP_CONFIG)
  );
};

UserService.ɵprov = ɵɵdefineInjectable({
  token: UserService,
  factory: UserService.ɵfac,
  providedIn: 'root'
});

// Inheritance case: subclass reuses parent's ɵfac
class AdminUserService extends UserService {
  // no own constructor
}
AdminUserService.ɵfac = function AdminUserService_Factory(t) {
  return UserService.ɵfac(t || AdminUserService);
};
```

Key details this reveals:

- `@Optional()` compiles to an integer bitmask flag (`InjectFlags`), not a runtime decorator lookup — `Self`, `SkipSelf`, `Host`, `Optional` are all just bits OR'd together and passed as the second argument.
- `@Inject(APP_CONFIG)` simply changes *which token identifier* gets baked into the `ɵɵinject(...)` call; it has no runtime cost beyond that.
- Token identity is the resolution key everywhere — `HttpClient` the class, `APP_CONFIG` the

`InjectionToken` object — never a string.

3.2 Simplified reimplementation of the injector resolution walk

```
type Token = any;

const NOT_FOUND = Symbol('NOT_FOUND');

interface InjectorRecord {
  factory: () => any;
  value: any | typeof NOT_FOUND;
  multi?: boolean;
}

/** Stand-in for R3Injector (module/environment injector). */
class SimpleEnvironmentInjector {
  private records = new Map<Token, InjectorRecord>();

  constructor(
    providers: { provide: Token; useClass?: any; useFactory?: () => any; useValue?: any;
multi?: boolean }[],
    public parent: SimpleEnvironmentInjector | null = null
  ) {
    for (const p of providers) {
      const factory = p.useFactory
        ? p.useFactory
        : p.useValue !== undefined
        ? () => p.useValue
        : () => new p.useClass(...resolveDeps(p.useClass));
      this.records.set(p.provide, { factory, value: NOT_FOUND, multi: p.multi });
    }
  }

  get(token: Token, notFoundValue: any = THROW_IF_NOT_FOUND): any {
    const record = this.records.get(token);
    if (record) {
      if (record.value === NOT_FOUND) {
        record.value = record.factory(); // lazy instantiation, cached forever
      }
      return record.value;
    }

    // Tree-shakable provider: check token.ɵprov before climbing.
    const prov = token && token.ɵprov;
    if (prov && this.isRoot && matchesScope(prov.providedIn, this)) {
      const value = prov.factory();
      this.records.set(token, { factory: prov.factory, value });
      return value;
    }

    if (this.parent) {
      return this.parent.get(token, notFoundValue);
    }
  }
}
```

```

    if (notFoundValue !== THROW_IF_NOT_FOUND) return notFoundValue;
    throw new Error(`NullInjectorError: No provider for ${describeToken(token)}!`);
  }

  get isRoot() {
    return this.parent === null;
  }
}

/** Stand-in for NodeInjector: one per element with providers. */
class SimpleNodeInjector {
  constructor(
    private records: Map<Token, InjectorRecord>,
    private parentNode: SimpleNodeInjector | null,
    private hostBoundary: boolean, // true at a component's host element
    private moduleInjector: SimpleEnvironmentInjector
  ) {}

  get(token: Token, flags: { self?: boolean; skipSelf?: boolean; host?: boolean; optional?: boolean } = {}): any {
    let node: SimpleNodeInjector | null = flags.skipSelf ? this.parentNode : this;

    while (node) {
      const record = node.records.get(token);
      if (record) {
        if (record.value === NOT_FOUND) record.value = record.factory();
        return record.value;
      }
      if (flags.self) break; // @Self(): do not climb further
      if (flags.host && node.hostBoundary) break; // @Host(): stop after this level
      node = node.parentNode;
    }

    // Element tree exhausted (or restricted) -> hand off to module tree.
    if (!flags.self) {
      return this.moduleInjector.get(token, flags.optional ? null : THROW_IF_NOT_FOUND);
    }
    if (flags.optional) return null;
    throw new Error(`NullInjectorError: No provider for ${describeToken(token)}!`);
  }
}

const THROW_IF_NOT_FOUND = Symbol('THROW_IF_NOT_FOUND');
function resolveDeps(_ctor: any): any[] { return []; } // placeholder
function matchesScope(providedIn: string, injector: SimpleEnvironmentInjector) {
  return providedIn === 'root' && injector.isRoot;
}
function describeToken(t: Token) { return t?.name ?? t?.toString?.() ?? String(t); }

```

This mirrors the real shape closely: `NodeInjector.get` climbs element-level records first and only escalates to the environment injector once the element chain is exhausted or bounded by `@Host / @Self`; `R3Injector.get` caches lazily and consults `eproov` before climbing to its parent.

4. Internal Working

4.1 The dual-tree walk in detail

Concretely, in real Ivy:

- Each `LView` has a `TView` describing static shape, and injector data lives in the `TView.data / LView` slots for nodes that declared `providers / viewProviders`. A `NodeInjector` isn't a standalone object per node in the common case — it's a *view* of specific slots in the `LView / TView` arrays, addressed via `TNode` index math (`getParentInjectorLocation`, `getNodeInjectorLView`, etc.). This is a deliberate memory optimization: allocating a real `Injector` object per DOM node would be far too costly at scale.
- Walking "up" the element tree means walking `TNode.parent` (and, at component boundaries, jumping to the *parent view's* host `TNode` via `LView[HOST_NODE] / DECLARATION_VIEW`, so an embedded view created by `*ngIf / *ngFor` resolves against its **lexical/declaration** parent, not merely its container's runtime parent — this is what makes `@Host()` and structural directive scoping behave predictably).
- `viewProviders` are visible only to the component's own template (and its children), while `providers` (declared alongside `viewProviders`) are also visible to content projected in from outside — this asymmetry is implemented by which injector "slot set" a lookup is allowed to consult depending on whether the requester is inside the component's view or is a directive on the host element itself/projected content.
- Once the element chain bottoms out (no more `TNode.parent`, no more declaration-view jumps), resolution calls into the `EnvironmentInjector` recorded for that `LView` (`LView[INJECTOR]`), which is an `R3Injector`. `R3Injector.get` checks its own `records`, and on miss delegates to `this.parent.get(...)`, continuing until it reaches the platform injector and finally the `NullInjector`, which always throws (or returns the caller-supplied not-found value).
- Standalone components/lazy routes each may introduce their **own** `EnvironmentInjector` as a child of the app root's, created via `createEnvironmentInjector()`; that's why a lazily loaded standalone component's `providedIn: 'any' / module-scoped` providers can get a fresh instance per lazy boundary while `providedIn: 'root'` services remain singletons app-wide (cached on the *root* injector, not the lazy child).

4.2 The injection context stack `inject()` reads

There is effectively one active "current injector" at a time, held in a module-scoped variable in Ivy's DI internals (historically `_currentInjector`, wrapped today by helpers commonly named around

`setInjectorContext / setCurrentInjector` plus

`assertInInjectionContext / getCurrentInjectionContext`). The discipline is:

1. Before Ivy invokes any `@fac` factory (during first construction of a component/directive/service, or during a manual `injector.get(Token)` call), it captures the previous value, sets the new "current injector" to the injector doing the resolving, and defers restoration to a `finally`.
2. Inside the factory body, every `@inject(Token)` / user-called `inject(Token)` simply reads that

currently-installed injector and calls `.get(token)` on it — that's the entire mechanism. There's no separate "stack" object; correctness relies on factories always being synchronous, so save/restore around a single call is equivalent to a proper stack (LIFO order falls out naturally from nested synchronous calls).

3. `runInInjectionContext(injector, fn)` and `EnvironmentInjector.runInContext` are the *manual* equivalent of what Ivy does automatically around factory invocation — they let you call `inject()` safely from inside a `Promise.then`, an RxJS pipeline callback, a `setTimeout`, or a test's arrow function.
4. Angular also tracks a distinct "in Angular's reactive/DI-eligible zone" concept for things like `assertInInjectionContext` used by APIs such as `takeUntilDestroyed()` — that check is really just "is `getCurrentInjectionContext()` (i.e., the current-injector slot) non-null right now."

4.3 Lazy registration of `providedIn: 'root'`

Walking through it end to end:

1. At compile time, `UserService.ɵprov = { token: UserService, factory: UserService.ɵfac, providedIn: 'root' }` is baked in — nothing about this touches any injector yet.
2. At bootstrap, the root `R3Injector` is constructed with whatever explicit `providers` you passed to `bootstrapApplication / root NgModule`, plus a handful of platform-level records. `UserService` is **not** among them.
3. The first time some component does `inject(UserService)` (or has it as a constructor param), `NodeInjector` climbs the element tree, finds nothing, delegates to the root `R3Injector`.
4. `R3Injector.get(UserService)` misses its `records` map, checks `UserService.ɵprov.providedIn === 'root'`, confirms this injector is the root, invokes `UserService.ɵfac()` (which itself may recursively `inject()` further dependencies, each resolved through the very same mechanism), and stores the resulting instance into its `records` map keyed by `UserService`.
5. Every subsequent `inject(UserService)` anywhere in the app hits the now-populated `records` entry and returns the cached singleton — no further factory invocation.

This lazy-on-first-use + cache-forever pattern is also exactly why services with `providedIn: 'root'` that are never injected anywhere are eligible for tree-shaking: the bundler can prove the factory is never referenced by a live code path and drop it, whereas providers registered explicitly in an array are always retained because they were referenced directly.

5. Edge Cases & Gotchas

- `inject()` must be called from an injection context (NG0203) — thrown whenever the current-injector slot is empty at call time: calling `inject()` at the top of a module file, inside a `setTimeout / Promise` callback without `runInInjectionContext`, inside a plain (non-factory) method invoked later, or inside an RxJS operator callback that fires asynchronously after the constructor/factory has already returned and the context was restored to the previous (often `undefined`) value.
- **Field initializers vs. constructor body** — `private http = inject(HttpClient);` works because field initializers execute as part of the constructor, which itself runs while Ivy still has the injector installed; a method called later (e.g., from `ngOnInit` triggered by change detection, which is a different call stack entirely but is still within an active injector context for that component so `inject()` inside `ngOnInit`

also works) versus a callback fired from `addEventListener / setTimeout` (no active context) behaves very differently — the rule is "is this synchronously part of Angular's own construction/lifecycle call, or an externally-scheduled callback."

- **Circular dependency detection** — when `R3Injector.get(TokenA)` is mid-resolution (its factory is running) and that factory's dependency chain calls back into `get(TokenA)` again before the first call completed, Ivy detects this via a "currently resolving" marker: `R3Injector` maintains a resolving-stack (`NG_TEMP_TOKEN_PATH` style bookkeeping in older docs, effectively a "circular" sentinel value stored temporarily in the record while the factory executes). If the same token is requested again while its record is in that "being constructed" state, Angular throws `NG0200: Circular dependency in DI detected for <Token>` with a chain describing the path (e.g., `A -> B -> C -> A`).
 - **`forwardRef()` is a reference-timing fix, not a cycle-breaker at the DI-resolution level** — it solves the *different* problem of a class token not being defined yet at the point another class's decorator metadata references it (JS hoisting: `class B { constructor(@Inject(forwardRef(() => A)) a: A) {} } class A {}` where `A` is declared after `B` in the file). `forwardRef(() => A)` just wraps a function that's invoked lazily when the token value is actually needed, deferring the `A` identifier lookup past module-evaluation order. It does **not** let you have `A` depend on `B` which depends on `A` in the same constructor-injection instant — that's still a genuine `NG0200` circular dependency and will still throw, `forwardRef` or not. Genuine cycles are broken architecturally: injecting `Injector` itself and calling `.get()` lazily inside a method (deferring resolution past construction time), restructuring so one side depends on an abstraction/ `InjectionToken` provided by a factory that doesn't eagerly need the other instance, or splitting a service in two.
 - **Multi-provider construction order** — `{ provide: TOKEN, useClass: A, multi: true }` combined with further `{ provide: TOKEN, useClass: B, multi: true }` registrations produces an array whose element order is the **registration order encountered while building the injector's records**, which for a single injector is simply declaration order in the `providers` array, but across the module/environment injector chain, **each injector level contributes its own multi-array independently** — a child `EnvironmentInjector`'s multi-provider array for a token does *not* automatically merge with its parent's; if the token isn't re-declared as multi in the child, the child inherits (via normal chain resolution) the parent's full array unchanged; if the child *does* declare more multi providers for the same token, the child injector's `get()` returns only its own array (the parent's entries are shadowed, not concatenated) — a common source of "why did my `HTTP_INTERCEPTORS` from the lazy module replace instead of append to the root's" bugs.
 - **`@Optional()` returning null vs. `NullInjectorError`** — `@Optional()` only suppresses the *final* `NullInjector` throw; it does not suppress `NG0200` circular-dependency errors or other genuine construction-time exceptions thrown inside a dependency's own factory.
 - **`useExisting` vs `useClass` cycles** — `{ provide: A, useExisting: B }` is a pure alias (no new instance, just redirects the token lookup) and cannot itself introduce a cycle the way `useClass / useFactory` chains can, since it never invokes a constructor.
-

6. Interview Questions & Answers

Q1. What are `ɵfac` and `ɵprov`, and which one does what?

`ɵfac` is the compiled factory function that knows how to construct one instance of a class, including which tokens to pass to its constructor. `ɵprov` is the provider *definition* — it says under what conditions (`providedIn`) that factory should be registered as the default provider for the token if nothing else provides it explicitly. `ɵfac` answers "how do I build one," `ɵprov` answers "where/whether I should be auto-registered."

Q2. Why doesn't Angular need `reflect-metadata` for constructor injection after Ivy?

Because the compiler resolves constructor parameter types statically at compile time (via the TypeScript AST/type checker) and emits literal `ɵinject(Token)` calls directly into the generated `ɵfac` function body. There's no runtime step that needs to re-discover "what type is parameter 2" — it's already hardcoded as executable code, so `Reflect.getMetadata('design:paramtypes', ...)` is never called by Angular's runtime.

Interviewer intent: distinguishes candidates who've internalized Ivy's compile-time codegen model from those repeating "Angular uses decorators + `reflect-metadata` for DI," which was true pre-Ivy/for other frameworks but is not how modern Angular's runtime works.

Q3. Walk me through what happens, step by step, when a component injects a service via constructor.

The compiler emitted a `ɵfac` for the component that includes `ɵinject(MyService)` as one of its arguments. When Ivy instantiates the component (creating its `LView`), it sets the current-injector context to that component's `NodeInjector` view, invokes the component's `ɵfac`, which calls `ɵinject(MyService)`. That call reads the currently-installed injector, asks it for `MyService`: the `NodeInjector` climbs the element tree looking for an explicit `providers / viewProviders` entry; finding none, it delegates to the associated `EnvironmentInjector`; that injector checks its own records, misses, checks `MyService.ɵprov.providedIn`, matches `'root'`, invokes `MyService.ɵfac()` (recursively resolving *its* dependencies the same way), caches the instance, and returns it up the chain into the component's constructor argument.

Q4. What's the difference between `NodeInjector` and `ModuleInjector / EnvironmentInjector`, and why does Angular have two trees instead of one?

`NodeInjector` mirrors the DOM/component tree and resolves component/directive-level `providers / viewProviders`; it's cheap per-node because it's really just indexed access into existing `LView / TView` arrays, not a real allocated object per node. `EnvironmentInjector / ModuleInjector` mirrors module/standalone-injector nesting (root, lazy feature injectors) and is a real object graph (`R3Injector` instances with `.parent` pointers) because module-level providers are far fewer and allocation cost doesn't matter there. Keeping them separate lets element-scoped providers (e.g., a `providers: []` on one specific component instance) not pollute or need representation in the app-wide module tree, while `providedIn: 'root'` singletons don't need per-element storage.

Q5. How does `inject()` know which injector to use — is it just reading this?

No — `inject()` has no receiver at all; it's a free function. It reads a single mutable "current injector" slot that Ivy sets right before invoking any factory function (and restores right after, in a `finally`). That's why `inject()` only works synchronously during construction/factory execution (or inside `runInInjectionContext`) — outside that window, the slot is empty and `inject()` throws `NG0203`.

Q6. Why does calling `inject()` inside a `setTimeout` callback throw, even if it's inside `ngOnInit`?

Because by the time the `setTimeout` callback actually runs, the synchronous call stack that had the injector installed has long since returned, and Ivy already restored the "current injector" slot to whatever it was before (typically empty/undefined at the top level). `inject()` isn't scoped by which component's file you're in — it's

scoped by whether an injector is *currently* installed on the call stack *right now*. Fix: capture `inject()`'s result before scheduling the callback, or wrap the callback body in `runInInjectionContext(injector, () => ...)`.

Interviewer intent: checks whether the candidate understands "injection context" as an ambient, time-bounded, stack-based mechanism rather than something tied to lexical scope or component identity.

Q7. How is a circular dependency between two injectables detected, and what error do you get?

When an injector's `get()` starts constructing a token's value (invoking its factory), it marks that record as "currently resolving." If, before that factory call returns, the same injector is asked to resolve the same token again (because some dependency deep in the chain depends back on it), the injector sees the "currently resolving" marker and throws `NG0200: Circular dependency detected`, with a description of the resolution path (e.g., `ServiceA -> ServiceB -> ServiceA`).

Q8. Does `forwardRef()` fix circular DI dependencies?

No — it fixes a *different*, narrower problem: referencing a class token before its `class` declaration has executed (TDZ/hoisting order within a module), typically when two classes in the same file reference each other in decorator metadata. `forwardRef(() => A)` just defers evaluating the `A` identifier until it's actually needed (lazily, inside a function), so it works around declaration order. It cannot let `A`'s constructor and `B`'s constructor mutually depend on fully-constructed instances of each other — that's a genuine `NG0200` cycle and will still throw regardless of `forwardRef`. True cycles require an architectural fix: inject `Injector` and resolve one side lazily inside a method rather than the constructor, introduce an abstraction/token that breaks the direct class-to-class dependency, or merge/split responsibilities.

Interviewer intent: many candidates conflate "forwardRef solves circular DI" with the actual, narrower truth; this question filters for people who've hit `NG0200` in practice versus people repeating docs from memory.

Q9. Why is a `providedIn: 'root'` service not instantiated at application bootstrap?

Because root-scoped registration is lazy: the compiler only attaches `@prov` metadata to the class describing *how* it would be provided; the root `EnvironmentInjector`'s `records` map has no entry for it until the first `get()` call for that token misses and falls through to the `@prov.providedIn === 'root'` check, at which point the factory runs once and the result is cached. If nothing in the app ever injects it, the factory never runs, and (assuming no side-effect-only imports keep it alive) a bundler can tree-shake the class out entirely.

Q10. If a lazy-loaded standalone route provides its own version of a `providedIn: 'root'` service in its own `providers` array, which instance does a component in that lazy chunk get, versus a component elsewhere in the app?

Components resolved within the lazy `EnvironmentInjector`'s subtree get the lazy chunk's own instance (found in that child injector's own `records`, found before the walk would ever reach the root injector's cached singleton). Components elsewhere in the app, resolving through injectors that never delegate into that lazy child, still get the root-cached singleton. This is exactly how a lazy route can locally override an otherwise app-wide singleton without needing `providedIn: 'any'` gymnastics — the child injector's own explicit provider always shadows a parent match once found.

Q11. What determines multi-provider array ordering when the same multi-token is provided at both a parent and a child `EnvironmentInjector`?

Each injector level's multi-provider array is independent — it's built purely from declarations registered directly *at that injector*, in declaration order. If a child injector declares more multi-providers for a token that the parent also declares as multi, the child's `get()` returns only the child's own array (its own registrations), not a concatenation with the parent's; the parent's entries are shadowed entirely, exactly like a non-multi override,

just packaged as an array. To truly append rather than replace, you'd need to re-provide the parent's tokens again explicitly in the child (or restructure to a single injector level).

Q12. What role does `@Optional()` actually play in the resolution algorithm, mechanically?

It changes what happens at the very last step, when `NullInjector` (or an equivalent "not found" terminal case) would otherwise be reached: instead of throwing `NullInjectorError / NG0201`, the resolution returns `null`. It has no effect at any earlier stage — it does not suppress a genuine circular-dependency error, does not change traversal order, and does not affect whether a provider is found; it purely governs the final failure behavior.

Q13. Why can't Angular use string-keyed tokens the way some older/simpler DI containers do?

Because token identity in Ivy DI is by object/class reference equality (`Map`-keyed by the actual `Token` value, e.g. the class function object or the `InjectionToken` instance), not by name. This survives minification (property/class renaming doesn't change object identity) and avoids accidental collisions between unrelated tokens that happen to share a string name across different files/libraries — an `InjectionToken<T>` (`'CONFIG'`) in library X and one in library Y with the same description string are still distinct token objects and won't collide.

Q14. What's the practical difference in when a `NodeInjector` record and an `R3Injector` record get created, and why does that matter for performance?

`NodeInjector` "records" aren't separately-allocated objects at all in the common case — they're indices into arrays that already exist as part of a component's `LView / TView` (allocated once when the view is created, whether or not that component declares its own `providers`). `R3Injector` records are entries in a real `Map`, lazily populated on first successful resolution and cached forever afterward. This matters because per-element injector lookups happen far more often (once per node in a large template) than module-level lookups, so Ivy specifically avoids a real object allocation per DOM node, trading that for index arithmetic into pre-existing view data structures — a deliberate memory/perf tradeoff baked into the two-tree design.

7. Quick Revision Cheat Sheet

- `efac` = compiled factory function; knows how to build one instance and what tokens its constructor needs — generated statically from resolved constructor types, not via runtime reflection.
- `eprovid` = compiled provider definition; declares `token`, `factory` (usually `= ClassName.efac`), and `providedIn` scope for auto-registration.
- **No `reflect-metadata`**: TypeScript AST → compiler statically bakes `ɵinject(Token)` calls directly into generated JS. Nothing to reflect at runtime.
- **Two trees**: `NodeInjector` (element/component tree, index-based into `LView / TView`, no per-node allocation) walked first; falls through to `EnvironmentInjector / ModuleInjector` (`R3Injector` object graph with real `.parent` pointers) only once the element chain is exhausted.
- **Resolution order**: current element → ancestor elements (respecting `Self / SkipSelf / Host`) → component's `EnvironmentInjector` → parent environment injectors → platform injector → `NullInjector` (throws `NG0201` unless `Optional`).
- **`inject()` context**: a single mutable "current injector" slot set right before factory invocation, restored in `finally` right after. `runInInjectionContext()` is the manual equivalent for async callbacks. Empty slot → `NG0203`.
- **`providedIn: 'root'`**: registered lazily — first `get()` miss on the root injector triggers

`@prov.providedIn` check → factory runs once → result cached on that injector forever. Unused services are tree-shakable.

- **Circular DI (NG0200)**: detected via a "currently resolving" marker on the record being constructed; re-entrant `get()` for the same token mid-construction throws with the dependency path.
- `forwardRef()`: fixes forward-reference/declaration-order problems only (lazy identifier lookup); does **not** and cannot resolve genuine mutual-construction cycles — those still throw `NG0200`.
- **Multi-providers**: each injector level's multi array is independent and self-contained; a child re-declaring the same multi-token shadows (replaces), never concatenates with, the parent's array.
- `@optional()`: only changes the terminal `NullInjector` behavior (`null` instead of throw); irrelevant to circular-dependency errors or mid-chain failures.
- **Token identity**: by reference (class or `InjectionToken` object), never by string — survives minification, avoids name collisions across libraries.

Created By - Durgesh Singh